

Digital British Coin (DBC) Node

Reference implementation — Whitepaper v1.0 (2026)

Rust full-node prototype aligned with the DBC whitepaper: CPU-oriented proof-of-work, Schnorr signatures, Bech32m addresses, UTXO model, uncle blocks, and libp2p networking.

Download & launch

- Windows release zip: [Google Drive](#)
 - Launch details (genesis, bootstrap, verify): [docs/LAUNCH.md](#)
 - Public whitepaper PDF: [docs/DBC Whitepaper Public.pdf](#)
-

Features (whitepaper alignment)

Whitepaper item	Status
Total supply 42,000,000 DBC	Implemented
Block time 15 minutes (target)	Implemented (difficulty adjustment)
Halving every 420,000 blocks (~8 years)	Implemented
Schnorr signatures (secp256k1)	Implemented
Bech32m dbc1 addresses (SHA3 + BLAKE3)	Implemented
BLAKE3 block / transaction hashing	Implemented
BritishWork memory-hard PoW	Implemented (fixed 2048 MiB in release builds — consensus)
Uncle blocks (max 2, 7-block lookback, 75% reward)	Implemented
Difficulty retarget every 1,008 blocks (144-block WMA)	Implemented
Max block size 2 MB	Implemented

libp2p P2P, Noise encryption, GossipSub	Implemented
Kademlia DHT peer discovery	Implemented (/dbc/kad/1.0.0, on by default)
Community peer list (config/peers.txt)	Implemented (empty in official release)
UTXO set (RocksDB)	Implemented
Mempool + replace-by-fee (RBF)	Implemented
BIP-39 / BIP-32 HD wallet	Implemented
Genesis message (Times 27/May/2026)	Implemented
Anonymous fair-launch workflow	Documented below

Planned / not in this release: full RandomX-compatible BritishWork, SPV light clients, HTTP RPC, automated DNS seed resolver, parameter rotation for ASIC resistance.

Anonymous fair launch (no founder IP)

You can launch DBC **without miners connecting to you**. The official repo ships **no hardcoded seed IPs** and an **empty** config/peers.txt.

5-minute Quick Start (copy/paste)

If you just want to **run a node + mine**, follow this exactly.

Windows (PowerShell) — new miner

1) **Open PowerShell** in the folder with dbc-node.exe.

2) **Make a wallet (save the 24 words!)**

```
.\dbc-node.exe wallet-new
```

Copy the dbc1... address it prints (that is where mining rewards go).

3) **Start node (sync). Start mining using the UI toggle (do NOT run init).**

```
.\dbc-node.exe run --listen /ip4/0.0.0.0/tcp/8334 ` --bootstrap
/dns4/digitalbritishpound.duckdns.org/tcp/8333/p2p/12D3KooWAmFcBBRh2H2SQQ5u2b2LU57kAToYKx18xct5zh3
```

Open the DBC **Wallet/Miner UI** from the Windows installer:

- Set your **payout address** to your `dbc1...`
- Click **Start Miner**
- Click **Stop Miner** to pause

If you do not have the UI installed, the CLI fallback is to add `--mine --address dbc1PASTE_YOUR_ADDRESS_HERE`.

Leave it running. You will see blocks sync from height 0 and then it will start mining.

Linux / macOS — new miner

1) Make a wallet (save the 24 words!)

```
./dbc-node wallet-new
```

2) Start node (sync). Start mining using the UI toggle (do NOT run `init`).

```
./dbc-node run --listen /ip4/0.0.0.0/tcp/8334 \ --bootstrap  
/dns4/digitalbritishpound.duckdns.org/tcp/8333/p2p/12D3KooWAmFcBBRh2H2SQQ5u2b2LU57kAToYKx18xct5zh3
```

If you do not have the UI available, the CLI fallback is to add `--mine --address dbc1PASTE_YOUR_ADDRESS_HERE`.

Two rules everyone must follow

- **Never share your 24-word mnemonic.** Anyone with it can steal your coins.
- **Do NOT run `init`** unless you are intentionally creating a different chain. Normal users sync genesis (block 0) from the bootstrap seed.

Wallet basics (receive / balance / send)

DBC uses a Bitcoin-style wallet model:

- **Address (`dbc1...`):** public “receive” identifier. Safe to share.
- **Mnemonic (24 words):** your private key backup. **Never share** it.

Receive

To receive mining rewards or payments, give the other person your `dbc1...` **address** (from `wallet-new`). That’s it.

View your balance

This reads the local chain/UTXO database and shows:

- **confirmed**: total received to the address
- **spendable**: confirmed but excluding immature coinbase (needs 100 blocks)

Run (replace the address):

```
dbc-node.exe --data-dir ./data balance --address dbc1PASTE_YOUR_ADDRESS_HERE
```

Include immature coinbase too:

```
dbc-node.exe --data-dir ./data balance --address dbc1PASTE_YOUR_ADDRESS_HERE  
--include-immature
```

Note: you cannot query balance while the node is running on the same `--data-dir` (RocksDB lock). Stop the node briefly, run `balance`, then restart.

Send

Sending creates a signed transaction from your address to another address.

```
dbc-node.exe --data-dir ./data send --from-mnemonic "word1 word2 ... word24" --to  
dbc1RECIPIENT --amount-dbc 1 --fee-dbc 0
```

Important:

- The mnemonic on the command line may be saved in terminal history. Treat it carefully.
- Coinbase rewards are **not spendable** until 100 blocks after they are mined.

What the founder publishes (safe)

1. **Release binary** (or source + build instructions) and its SHA-256 hash.
2. **Genesis block hash** and `genesis.json` from `export-genesis` (after mining genesis offline).
3. **Consensus constants** (already in this repo) — not your home IP, not your libp2p peer id.

Official genesis (this release)

- **Genesis hash**: 87f9442d436c6627f00a4bc025e149d0c2fe30dc5f77eb2c18acd086ba582a7d
- **Genesis file**: `genesis.json` (shipped with the release package)

Early miner bootstrap (first public node)

Use this multiaddr to join the network and sync block 0:

/dns4/digitalbritishpound.duckdns.org/tcp/8333/p2p/12D3KoowAmFcBBRh2H2SQQ5u2b2LU57kAToYKx18xct5zh3NV

If you are behind a home router, the seed operator must port-forward **TCP 8333** to the machine running `dbc-node`.

What the founder should NOT do

- Do **not** run `run --listen 0.0.0.0` on a home connection and tell everyone to `--bootstrap` your address.
- Do **not** commit your `multiaddr` to `config/peers.txt` in the main release.
- Do **not** post `peer-id` output tied to your identity if you want to stay anonymous.

How the network bootstraps without you

Mechanism	Role
Kademlia DHT (default)	Peers find each other once a few independent nodes are online.
Community seeds	Volunteers run nodes on VPS/Tor and post <code>multiaddrs</code> in forums or their own <code>peers.txt</code> forks.
<code>--bootstrap / peers file</code>	Users paste community <code>multiaddrs</code> ; never required to be the founder.
<code>--mdns</code>	Optional LAN dev only; off by default .

Founder workflow (offline genesis, no public node)

```
# 1. Wallet (offline machine is fine) cargo run --release -- wallet-new # Save mnemonic locally only. # 2. Mine genesis locally – no P2P (chain creator only) cargo run --release -- init --force-new-chain --address dbc1YOURADDRESS # 3. Export for forum post (hash + JSON only) cargo run --release -- export-genesis --out genesis.json # 4. Publish: binary hash, genesis hash, genesis.json # Do NOT publish your IP or peer id.
```

Miner / community node (independent of founder)

```
# Verify genesis hash matches the forum post, then sync from peers. # IMPORTANT: do NOT run `init` (that would create a different chain). # # First node you know about (bootstrap) will serve block 0 over P2P. cargo run --release -- run --listen /ip4/0.0.0.0/tcp/8333 \ --bootstrap /dns4/digitalbritishpound.duckdns.org/tcp/8333/p2p/12D3KooWAmFcBBRh2H2SQQ5u2b2LU57kAToYKx18xct5zh3 # Optional: community multiaddrs (from forum, not founder home IP) cargo run --release -- run --listen /ip4/0.0.0.0/tcp/8333 \ --bootstrap /ip4/VPS.IP/tcp/8333/p2p/COMMUNITY_PEER_ID \ --mine --address dbc1YOURADDRESS # Or maintain config/peers.txt with community lines (see config/peers.txt comments)
```

Community seeds that **choose** to be public run on a neutral VPS, then share:

```
cargo run --release -- run --listen /ip4/0.0.0.0/tcp/8333 # Log shows: local peer id +  
listening multiaddr – post those, not the founder's.
```

Requirements

- **Rust 1.70+** ([rustup](#))
- Windows, Linux, or macOS
- ~500 MB disk for build artifacts; RocksDB grows with chain usage

Build

```
cargo build --release
```

Binary: `target/release/dbc-node` (Windows: `target\release\dbc-node.exe`).

Quick start

```
# 1. Create a wallet (save the mnemonic) cargo run -- wallet-new # 2. Initialize  
genesis (block 0, BritishWork PoW) – chain creator only # Normal users must NOT run  
this (it would create a different chain). cargo run -- init --force-new-chain  
--address dbc1YOURADDRESS # 3. Export genesis for launch announcement (no network)  
cargo run -- export-genesis --out genesis.json # 4. Mine blocks (solo, no P2P) cargo  
run -- mine --blocks 10 --address dbc1YOURADDRESS # 5. Chain status cargo run -- info  
# 6. Join network (sync from bootstrap). Start mining in the UI (or add --mine). cargo  
run -- run --listen /ip4/0.0.0.0/tcp/8333 \ --bootstrap  
/dns4/digitalbritishpound.duckdns.org/tcp/8333/p2p/12D3KooWAmFcBBRh2H2SQQ5u2b2LU57kAToYKx18xct5zh3
```

All commands accept `--data-dir ./data` (default).

CLI reference

Command	Description
<code>wallet-new</code>	Generate 24-word BIP-39 mnemonic and dbc1 address

<code>wallet-addr <mnemonic></code>	Derive address from mnemonic
<code>init [--address ADDR] [--timestamp UNIX]</code>	Mine and store genesis block (chain must be empty)
<code>export-genesis [--out genesis.json]</code>	Write genesis JSON + print hash (no P2P)
<code>peer-id</code>	Print libp2p peer id from <code>data/peer_key</code> (community seeds only)
<code>mine --blocks N [--address ADDR]</code>	Mine N blocks extending the tip
<code>send --from-mnemonic "... " --to dbc1... --amount-dbc N [--fee-dbc F]</code>	Build signed tx, mine block including it
<code>balance --address dbc1... [--include-immature]</code>	Show confirmed/spendable balance for an address
<code>info</code>	Print tip height/hash, next difficulty, mempool size
<code>run</code>	Start P2P node (see flags below)

run flags

Flag	Default	Description
<code>--listen</code>	<code>/ip4/0.0.0.0/tcp/8333</code>	libp2p listen multiaddr
<code>--bootstrap</code>	(none)	Repeatable community peer multiaddrs
<code>--peers-file</code>	<code>config/peers.txt</code>	Extra peers (empty in official release)
<code>--mine</code>	off	Mine blocks when tip advances
<code>--address</code>	auto	Coinbase payout when <code>--mine</code>
<code>--no-dht</code>	DHT on	Disable Kademlia (not recommended)

<code>--mdns</code>	off	Enable LAN mDNS (dev only)
---------------------	------------	----------------------------

Run a network node

Independent miner (sync first) (default — DHT on, mDNS off, no founder IP):

```
cargo run --release -- run --listen /ip4/0.0.0.0/tcp/8333 \ --bootstrap
/dns4/digitalbritishpound.duckdns.org/tcp/8333/p2p/12D3KooWAmFcBBRh2H2SQQ5u2b2LU57kAToYKx18xct5zh3
```

Then click **Start Miner** in the UI (or use `--mine --address ...` as a CLI fallback).

In the Windows package, the UI executable is `dbc-ui.exe`.

With community bootstrap (replace with volunteer VPS multiaddr from forum):

```
cargo run --release -- run --listen /ip4/0.0.0.0/tcp/8333 \ --bootstrap
/ip4/203.0.113.10/tcp/8333/p2p/12D3KooWCommunityPeerIdHere \
```

Then click **Start Miner** in the UI (or add `--mine --address ...`).

LAN development only:

```
cargo run -- run --listen /ip4/127.0.0.1/tcp/8333 --mdns
```

Gossip topics: `dbc/blocks/v1`, `dbc/txs/v1`, `dbc/sync/v1` (height-based block catch-up).

Discovery: Kademlia DHT (`/dbc/kad/1.0.0`) + optional `--bootstrap /config/peers.txt`. mDNS only with `--mdns`.

Environment variables

Variable	Default	Description
DBC_BRITISHWORK_MIB	—	Tests only. Release builds use fixed 2048 MiB (consensus).
RUST_LOG	—	Tracing filter, e.g. <code>dbc_node=info, libp2p=warn</code>

BritishWork PoW memory is fixed to **2048 MiB** in release builds so all nodes agree on valid blocks.

Data directory

```
data/ chain/ # RocksDB: blocks by hash, height index, tip utxo/ # RocksDB: UTXO set
peer_key # libp2p Ed25519 identity (created on first `run`)
```

Delete `data/` to reset the node (dev only).

Architecture

```
src/ consensus.rs # Supply, halving, difficulty, limits crypto/ british_work.rs #
Memory-hard PoW wallet.rs # BIP-39/32, Schnorr, bech32m types/ dbc_types.rs # Block,
tx, scripts, merkle utxo.rs # UTXO set + RocksDB node/ chain.rs # Block acceptance,
orphans, mempool miner.rs # Block assembly + PoW grind validation.rs # Scripts,
signatures, block rules uncles.rs # Uncle validation and rewards genesis.rs # Genesis
block builder + export mempool.rs # RBF mempool network/ p2p.rs # libp2p: GossipSub +
Kademlia + optional mDNS seeds.rs # Load config/peers.txt protocol.rs # Gossip message
types storage/ chaindb.rs # Block storage main.rs # CLI config/ peers.txt # Community
peers (empty at fair launch)
```

Consensus flow: `accept_block` → BritishWork PoW check → chain extension rules →
`validate_block` (merkle, uncles, coinbase cap) → UTXO `apply_block` → persist.

Addresses: Pay-to-address scripts (`0x14` + 20-byte hash). Smallest unit: 1 pence = 10███ DBC.

Tests

```
cargo test
```

38 unit/integration tests. BritishWork uses a fast path under `cfg(test)`.

Regenerate this document as PDF

```
python scripts/generate_readme_pdf.py
```

Output: `docs/DBC_Node_README.pdf`

Licence

MIT — open source, per whitepaper intent.

Digital British Coin — dbc1 — 42,000,000 — BritishWork